

Weather Alert System w/ Visualisation

By Wasil Kayani

Phase 1: CRS Breakdown and Weather Data Formats

- **CRS:** To accurately map jet's telemetry data based on actual latitude and longitude coordinates ie; EPSG:4326 (WGS 84)
<https://epsg.org/search/by-name/?query=4326&sessionkey=9x1excaao4>
For Flat map rendering we use EPSG:3857 which is also used by Google Maps
Correct telemetry data mapping is required before checking collision detection with web-pulled weather grids
- **Weather Data Formats:** In standard GIS format, including .shp for geometry, .dbf for attributes and .shx for indexing GeoJSON is however the standard now making a JSON file with geographical features and properties
<https://geojson.org/>
 - Ingestion of .shp files and then conversion to GeoJSON
 - Ray Casting is used as the logic definition of checking whether the jet has intersected the weather polygon yet or not, Triggers are then generated based on ODD/EVEN number of intersections. This is done via SHAPELY library.

Phase 2: Administrative Boundaries and Local Data Sourcing

- **Data Sourcing & Optimization:** Administrative country boundaries for Pakistan are sourced from GADM, which provides high-resolution spatial maps down to provincial and district levels.
https://gadm.org/download_country.html
 - Raw Shapefiles contain high vertex density (hundreds of thousands of points), which must be geoprocessed to prevent computational latency. The Douglas-Peucker simplification algorithm is used to reduce the vertex footprint while preserving exact territorial shapes for real-time applications.
https://docs.qgis.org/latest/en/docs/user_manual/processing_algs/qgis/vectorgeometry.html#simplify
 - Static Airspace boundary was done via GADM Pakistan's shapefile whilst Dynamic Weather Layer was done initially via mock data but then through OpenWeatherMap API.

File Infrastructure & Execution Pipeline:

- **Ingestion Mechanics:** Python's json and shapely.geometry.shape modules are utilized to transform raw coordinate data arrays straight from static storage into structured spatial memory objects.
<https://shapely.readthedocs.io/en/stable/manual.html#geometric-objects>
- **Airspace Ingestion Flow:** Raw .shp files are compiled, cleaned of unnecessary metadata, and re-projected to EPSG:4326 inside QGIS desktop software before being exported into a web-optimized .geojson array format.
<https://qgis.org/>
- **Cloud vs. Local Execution:** For initial development, files are loaded inside Google Colab. Localized operating without active network dependencies to maintain cryptographic and structural air-gapping.

Phase 3: Data Enrichment & Spatial Buffering

- **Point-to-Polygon Translation:**
Real-time APIs like OpenWeatherMap transmit localized point coordinate pairs via nested JSON structures. Because a tactical jet warning system requires spatial containment checking rather than localized coordinate matching, geometric buffering is applied.
<https://openweathermap.org/api>
- **Mathematical Operations:** Using shapely.geometry.Point.buffer(), a point representation of a meteorological center is mathematically extruded into a unified circular polygon using a predefined spatial threshold.
<https://shapely.readthedocs.io/en/stable/reference/shapely.Point.html#shapely.Point.buffer>
- **Metadata Encapsulation:** The resulting geometry is wrapped directly alongside numerical attributes like temperature, wind velocity, and systemic severity metrics inside a structured, locally stored GeoJSON feature collection for standalone spatial queries.
- **Local Asset Consumption:** The system deserializes the locally cached live_weather_zone.geojson structure back into a standard Python dictionary via json.load(). Telemetry streams originating from aircraft (Latitude/Longitude pairs mapped as shapely.geometry.Point) are evaluated against the rehydrated polygon using the .contains() boolean query. This step functions independently of an active internet connection, matching air-gapped system requirements.

- **Numerical Weather Prediction Models:**

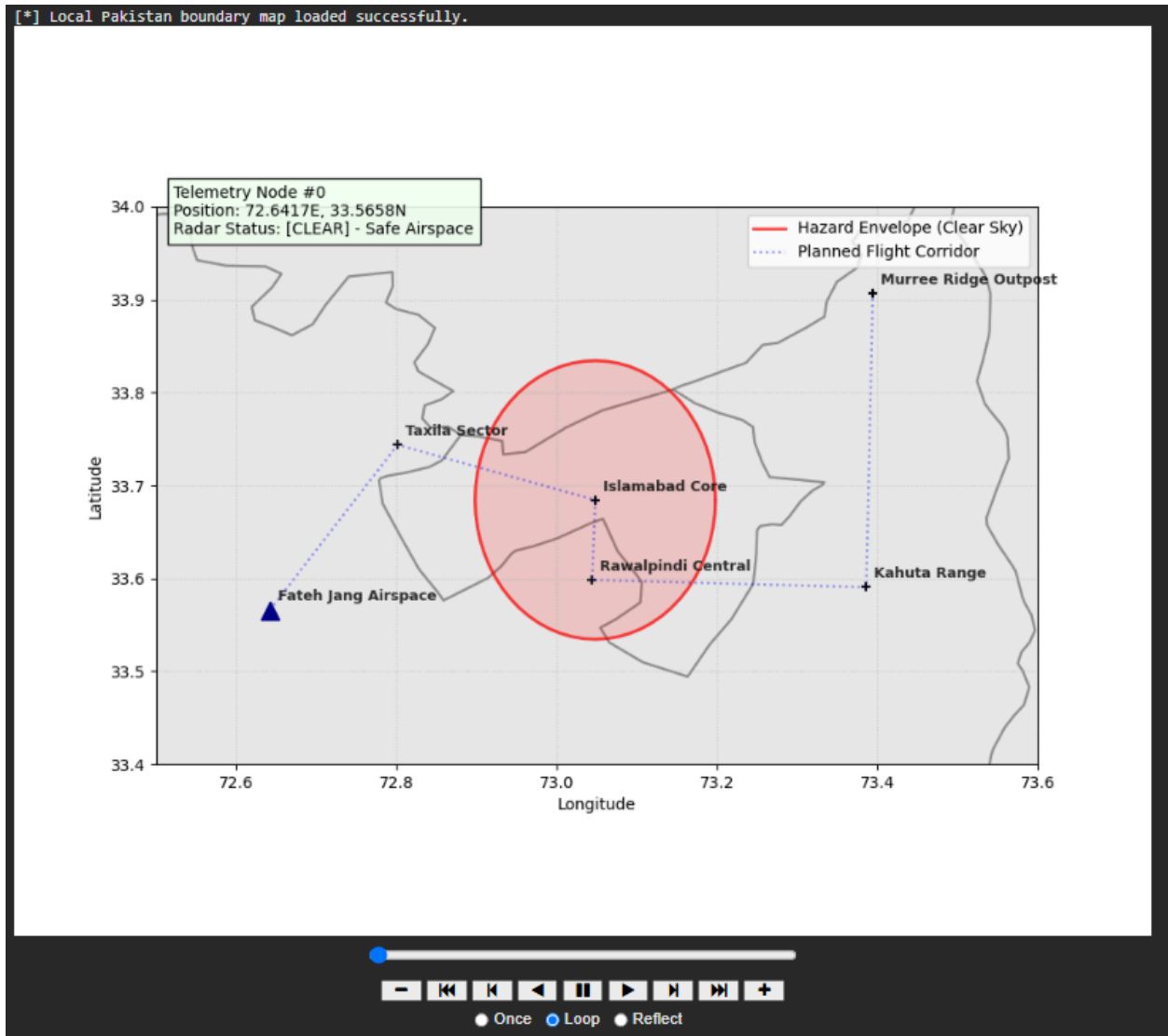
NWP Model	Spatial Scope	Grid Resolution	Update Frequency	Tactical Use Case in System	File Format
GFS	Global	~28 km	Every 6 hours	Long-range strategic flight planning; global baseline fallback.	GRIB2
ECMWF	Global	~9 km	Every 6 - 12 hours	High-accuracy continental routing; primary strategic layer.	GRIB / NetCDF
HRRR (or local eqv.)	Regional / National	~3 km	Every 1 hour	Real-time, tactical localized collision detection.	GRIB2

Phase 4: Tactical Visualization & Telemetry Tracking

- **Base Layer Mapping:** To provide spatial context without relying on web-based map tile servers, the system uses the geopandas library to render the local `pakistan_border.geojson` file as the static background canvas.
- **Live Radar Animation:** `matplotlib.animation` is utilized to create a localized radar loop. The system plots a sequenced flight corridor with regional waypoints (e.g., Fateh Jang, Murree Ridge) and animates a dynamic jet vector crossing the airspace.
- **Real-Time Execution:** Frame-by-frame, the radar loop executes the Phase 1 Point-in-Polygon logic. If the telemetry node intersects the active weather layer, the dashboard instantly triggers localized alphanumeric alerts (e.g., "CRITICAL BREACH") based on the geometry's metadata payload.

Phase 5: High-Resolution Rapid Refresh (HRRR) & Spatial Contouring

- **NWP Matrix Ingestion:** The architecture abandons API circular buffering in favor of ingesting raw GRIB2 meteorological matrices. The system parses this high-resolution data to extract exact physical grid values (e.g., wind velocity arrays).
- **Marching Squares Algorithm:** A spatial contouring engine (`skimage.measure.find_contours`) traces the exact, asymmetric boundaries of hazardous weather cells based on specific tactical thresholds.
- **Topological Normalization:** To prevent geometry crashes caused by grid-edge artifacts, the contour coordinate arrays are mathematically forced into closed `LinearRing` loops. The system filters out fragmented multi-polygons and self-intersections (`.buffer(0)`), ensuring only valid, true-to-life continuous geometries are serialized into the `live_weather_zone.geojson` payload for the collision engine.



Phase 6: AI-Driven Meteorological Prediction Models & Time-Series Pivot

- 1. SOTA Architecture Evaluations:

Traditional NWP vs. AI Models: Traditional Numerical Weather Prediction (NWP) models require massive supercomputer environments. AI models replace physical fluid dynamic equations with deep learning, reducing inference runtimes down to seconds on standard localized hardware.

Global SOTA Models: Architectures like GraphCast (Google DeepMind), Pangu-Weather (Huawei), and FourCastNet (Nvidia) achieve rapid global calculations but operate at coarse resolutions (~25km to 30km grid scales). This creates spatial

quantization issues, generating high false-positive collision rates for pinpoint tactical jet trajectories.

2. Refined Scope: 5 Provincial Capitals Ingestion

To eliminate spatial grid complexities and focus strictly on deterministic tactical nowcasting, the system design limits predictive metrics to the 5 provincial capitals of Pakistan: Karachi, Lahore, Peshawar, Quetta, and Gilgit. This changes the core problem structure from 2D spatial video prediction into Multivariate Time-Series Forecasting.

3. Dataset Sourcing & Repository Selection

- **Explored Repositories & Sources:**
- Open-Meteo Historical Weather API (ERA5 Reanalysis Baseline): <https://open-meteo.com/en/docs/historical-weather-api>
- Meteostat Weather Library: <https://meteostat.net/en/>
- AWS Labs GluonTS Time-Series Hub: <https://github.com/aws-labs/gluon-ts>
- Unit8 Darts Time-Series Repository: <https://github.com/unit8co/darts>

Selected Pipeline Implementation:

- **Dataset Selection:**
- Open-Meteo Historical API (ERA5). Chosen over raw local station metrics because it bridges reporting drops with gapless, consistent 10-year historical weather blocks (2014-2024), yielding clean training arrays for neural processing layers.
- **Model Selection:**
- Multivariate Long Short-Term Memory (LSTM) / Temporal Fusion Transformer (TFT) deployed using the Unit8 Darts ecosystem. Darts was selected as the operational pipeline repository due to its standardized API wrapping, enabling robust multi-model performance benchmarking across all 5 capitals simultaneously with minimized structural codebase modifications.
- **Model Architecture:**
 - **Type:** Recurrent Neural Network (RNN) configured specifically as an **LSTM**.
 - **Hidden Dimensions:** 32 units with 2 layers (`n_rnn_layers=2`), incorporating a 0.2 dropout rate to prevent overfitting.
 - **Temporal Window:** Configured with a `training_length` and `input_chunk_length` of **24 hours**. This means the AI looks at the previous 24 hours of sequential weather patterns to predict the next hour.

- **Historical Baseline:** 10 years of gapless hourly reanalysis data (2014–2024) sourced from **Open-Meteo (ERA5)**.

- Why was this choice optimal?

```
[*] Loading 30-year dataset into memory...
[*] Converting tabular data into Darts TimeSeries groups...
INFO:pytorch_lightning.utilities.rank_zero:GPU available: False, used: False
INFO:pytorch_lightning.utilities.rank_zero:TPU available: False, using: 0 TPU cores
INFO:pytorch_lightning.utilities.rank_zero:MPS available: False, using: 0 MPS
INFO:pytorch_lightning.callbacks.model_summary:
| Name | Type | Params | Mode
-----|-----|-----|-----
0 | criterion | MSELoss | 0 | train
1 | train_criterion | MSELoss | 0 | train
2 | val_criterion | MSELoss | 0 | train
3 | train_metrics | MetricCollection | 0 | train
4 | val_metrics | MetricCollection | 0 | train
5 | rnn | LSTM | 133.2 K | train
6 | lv | Linear | 132 | train
-----|-----|-----|-----
13.4 K Trainable params
0 Non-trainable params
13.4 K Total params
0.854 Total estimated model params size (MB)
7 Modules in train mode
0 Modules in eval mode
[*] Initializing Scaler & normalizing data...
[*] Compiling the LSTM Neural Network Architecture...
[*] Beginning training phase across all 5 provincial capitals...
/usr/local/lib/python3.12/dist-packages/pytorch_lightning/utilities/pytree.py:21: 'instance(treespec, LeafSpec)' is deprecated, use 'instance(treespec, TreeSpec) and treespec.is_leaf()' instead.
/usr/local/lib/python3.12/dist-packages/torch/utils/data/dataloader.py:775: UserWarning: 'pin_memory' argument is set as true but no accelerator is found, then device pinned memory won't be used.
  super().__init__(loader)

Epoch 0: 29% ██████████ 3902/13581 [03:00:27.28, 21.590s/train, loss=0.0021]
```

- **Data Alignment:** Loaded ten years of hourly meteorological data across five provincial capitals and stripped timezone metadata to prevent temporal alignment errors.

- **Preprocessing & Partitioning:** Converted the dataset into Darts TimeSeries objects, normalized the features using min-max scaling, and partitioned the data (withholding the final 720 hours as a blind test set).
- **Architecture & GPU Training:** Configured a 2-layer LSTM (32 hidden dimensions, 0.2 dropout) and optimized training on a GPU accelerator over 5 epochs to prevent browser timeouts.
- **Serialization:** Saved the final network weights (.pt and .pt.ckpt) and compiled them into a compressed deployment package (.zip) to preserve structural integrity.

Phase 7: MLOps Integration & Full-Stack Deployment

- **Shift to Dynamic MLOps:** Transitioned the predictive engine from a static, local inference script into an enterprise-grade Continuous Integration/Continuous Deployment (CI/CD) architecture.
- **System Decoupling:** Separated the architecture into three independent layers to ensure zero-latency tactical reporting: a Continuous Training (CT) background pipeline, an ASGI microservice backend, and a client-side visualization frontend.
- **Multivariate Hazard Logic:** Engineered a real-time evaluation matrix that continuously monitors 4 key environmental metrics against defined aviation safety thresholds, categorizing localized threats (e.g., High Wind, Extreme Heat) using flat boolean logic rather than nested loops.

Pipeline Implementation:

- **Backend Microservice (FastAPI & Uvicorn):** Developed a high-performance REST API that serves as the inference engine. Upon receiving a client request, it executes an automated ETL (Extract, Transform, Load) function—pinging the Open-Meteo API for the sector's previous 24 hours of live weather data, formatting it into Darts TimeSeries objects, running it through the LSTM .pt weights, and returning a JSON payload in milliseconds.
- **Continuous Training (CT) Pipeline:** Built a standalone Python automation script designed to routinely fetch the latest 14 days of meteorological telemetry and dynamically retrain/fine-tune the PyTorch model weights, preventing model degradation over time.
- **Tactical Frontend (React & Vite):** Deployed a dark-mode, responsive web interface utilizing Tailwind CSS v4.
- **Geospatial & Temporal Visualization:** Integrated react-leaflet to map custom national boundary .geojson files, plotting dynamic sector nodes that visually trigger upon hazard detection. Paired with recharts to render real-time, 12-hour predictive wind/temperature vectors alongside complete multivariate telemetry logs.

